

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Отчёт
о производственной, научно исследовательской работе

по теме:
«Разработка алгоритмов управления
манипулятором на основе методов
машинного обучения»

Работу
выполнил:
Сухов Н.М.
Группа: R3335
Преподаватель:
Краснов А.Ю.

Санкт-Петербург
2026

Содержание

Введение	3
1. Теоретическая постановка задачи и первое приближение решения	3
1.1. Постановка задачи	3
1.2. Движение к заданным точкам, градиентный спуск	4
1.3. Решение на Python	6
1.4. Учёт препятствий и проверка безопасности	7
2. Подготовка данных для нейросетевого оценивания	8
2.1. Предварительный поиск допустимых конечных конфигураций	9
2.2. Формирование входного вектора MLP	9
2.3. Экспертная оценка кандидатного шага	10
3. Обучение нейросетевой модели	11
3.1. Формирование обучающей выборки	11
3.2. Архитектура и параметры обучения	11
3.3. Результаты обучения	12
4. Планирование движения с использованием MLP	12
4.1. Генерация и фильтрация кандидатных шагов	12
4.2. Выбор шага и критерий остановки	13
5. Демонстрационные сцены и оценка результатов	13
5.1. Визуальная проверка траекторий	14
5.2. Диагностика ошибки и безопасных зазоров	15
Заключение	15

Введение

В рамках работы рассматривается задача управления плоским многозвенным роботом-манипулятором в двумерной рабочей области. Исходными данными для решения являются длины звеньев манипулятора, начальная конфигурация, целевые точки для выбранных сочленений, а также параметры препятствий, расположенных в рабочем пространстве. Управление сводится к подбору такого вектора углов, при котором выбранные сочленения достигают заданных точек, а траектория движения остаётся безопасной относительно внешних препятствий и самопересечений манипулятора.

Актуальность работы связана с тем, что при автоматическом управлении манипулятором недостаточно решить только задачу обратной кинематики. Даже если конечная конфигурация достигает целевых точек, переход к ней может проходить через запрещённые положения, в которых звенья пересекают препятствия или друг друга. Поэтому в работе последовательно рассматриваются два уровня задачи: первое приближение в виде градиентного решения обратной кинематики и расширенная постановка, в которой учитываются ограничения безопасности и используется нейросетевая модель для оценки локальных шагов движения.

Цель работы состоит в разработке и программной проверке алгоритма управления плоским многозвенным манипулятором на основе аналитической кинематики, градиентного спуска и нейросетевой оценки кандидатных управляющих воздействий.

Для достижения цели были поставлены следующие задачи:

- сформулировать математическую модель прямой кинематики манипулятора;
- записать функцию ошибки достижения целевых точек и аналитический градиент для её минимизации;
- реализовать первое приближение решения задачи методом градиентного спуска;
- ввести формальные критерии безопасности конфигурации и перехода между конфигурациями;
- сформировать входной вектор и экспертную оценку качества шага для обучения MLP;
- обучить нейросетевую модель и проверить её работу на демонстрационных сценах с препятствиями.

1. Теоретическая постановка задачи и первое приближение решения

1.1. Постановка задачи

Пусть манипулятор имеет n сочленений. Обозначим вектор углов поворота сочленений манипулятора: $\theta = (\theta_1, \theta_2, \dots, \theta_n) \in \mathbb{R}^n$ и вектор пар координат сочленений: $J = (J_1, J_2, \dots, J_n) \in (\mathbb{R} \times \mathbb{R})^n$, $J_m = (x_m, y_m)$. Функция решения прямой кинематики для робота с заранее заданными длинами звеньев:

$f_{fk} : \mathbb{R}^n \rightarrow (\mathbb{R}^2)^n$, $J = f_{fk}(\theta)$.

Необходимо перевести робота в заданное положение, выполнив обход расположенных в пространстве оперирования робота препятствий. Схема четырёхзвенного робота-манипулятора приведена на рисунке 1.1.

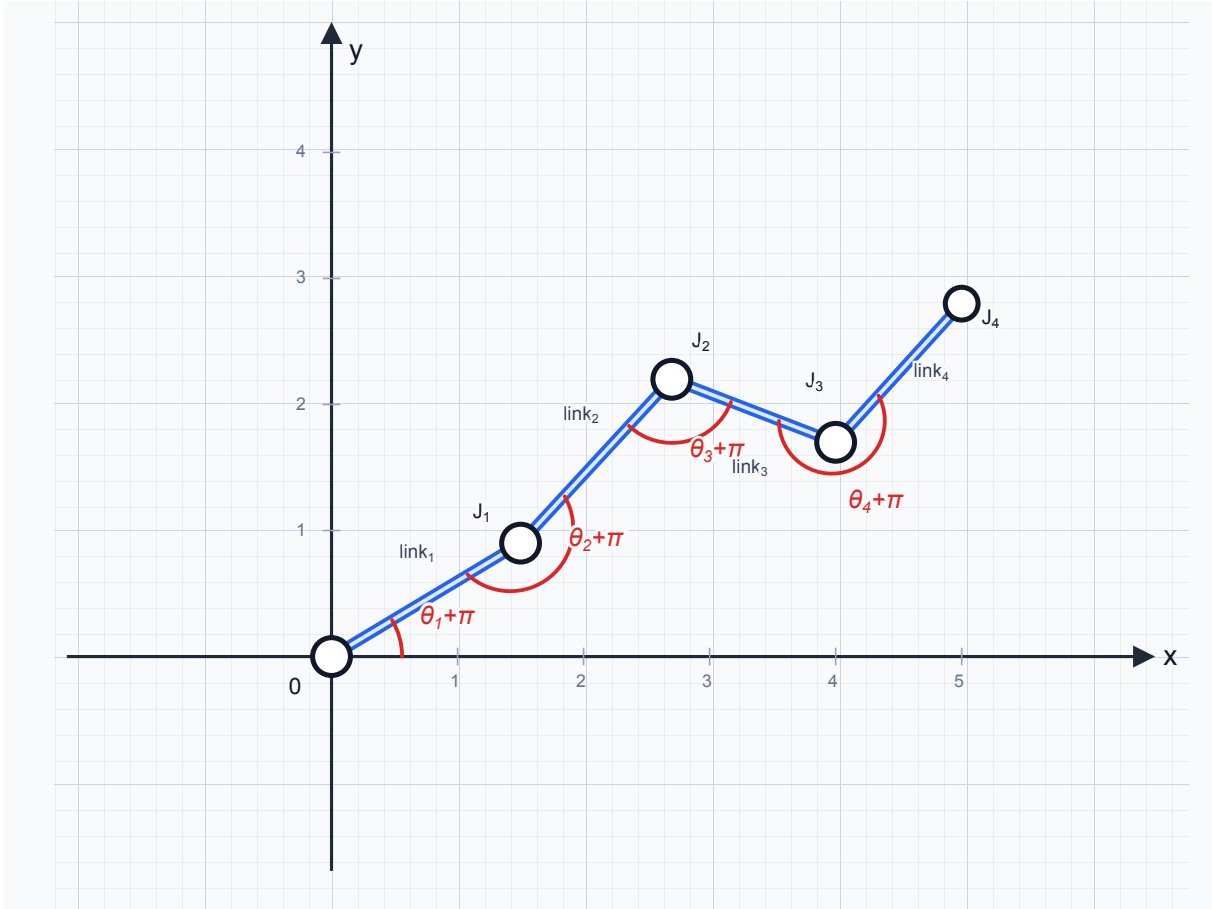


Рисунок 1.1. Схема четырёхзвенного робота-манипулятора

1.2. Движение к заданным точкам, градиентный спуск

Обозначим также целевые точки G_m , в которые должны прийти определённые сочленения робота (не все, а, например, только конечные). Индексы $I \subseteq \{1, \dots, n\}$ соответствуют номерам сочленений, к которым относятся целевые точки: $G_m = (x_m^*, y_m^*) \in \mathbb{R}^2$, $m \in I$. И расстояния между целевыми точками и сочленениями: $l_m(\theta) = \|J_m(\theta) - G_m\|_2$, $m \in I$, $L = \{l_m \mid m \in I\}$. Если $m, m+1 \in I$, то $\|G_{m+1} - G_m\|_2 = d_m$, где d_m — длина звена между J_m и J_{m+1} . Схема целевых точек и расстояний для сочленений показана на рисунке 1.2.

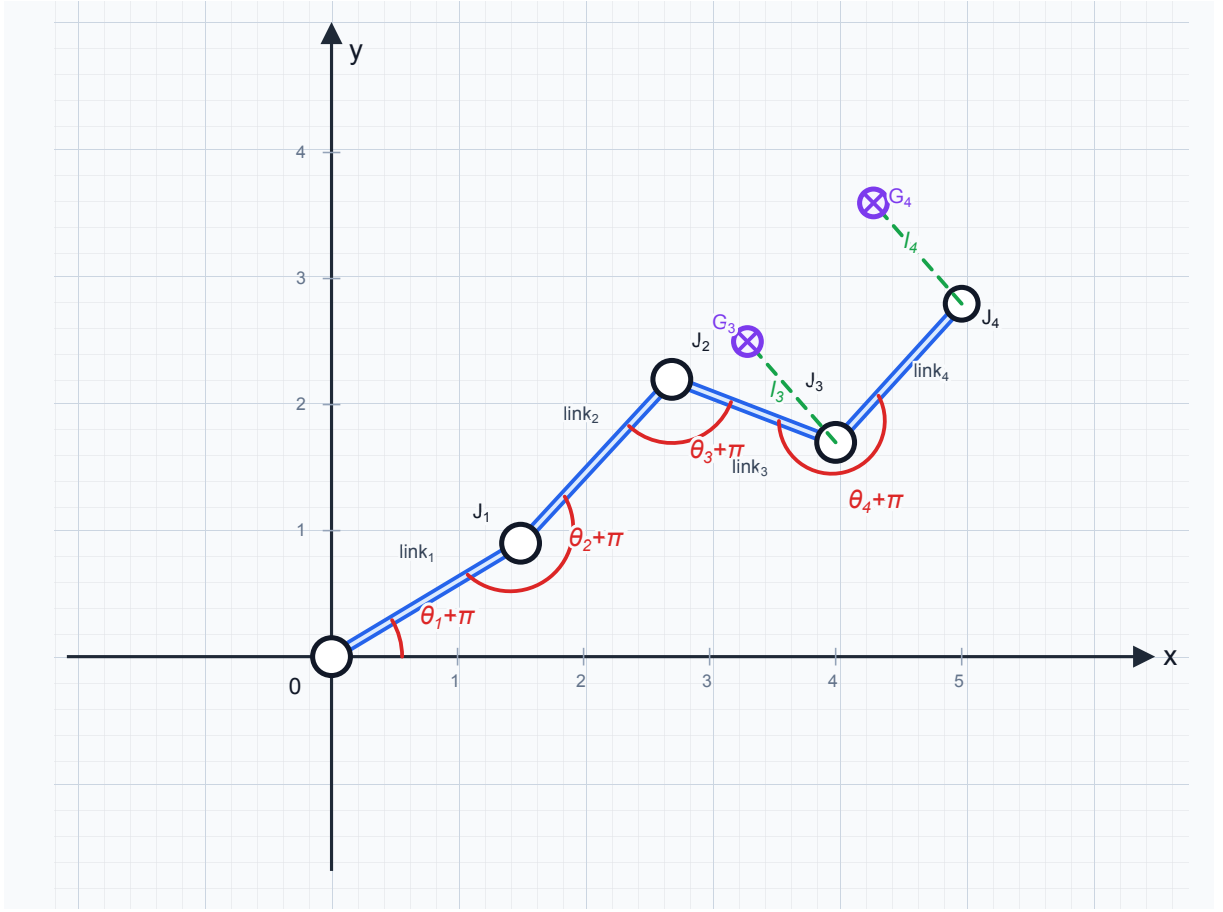


Рисунок 1.2. Схема целевых точек и расстояний для сочленений

Решение задачи регрессии будет заключаться в минимизации функции суммы расстояний:

$$f_{dist} : \mathbb{R}^n \rightarrow \mathbb{R},$$

$$f_{dist}(\theta) = \sum_{i \in I} \|(f_{fk}(\theta))_i - G_i\|_2^2 = \sum_{i \in I} ((f_{fk}(\theta))_i - G_i)^T ((f_{fk}(\theta))_i - G_i)$$

$$\theta^* = \arg \min_{\theta \in \mathbb{R}^n} f_{dist}(\theta)$$

Тогда:

$$\nabla_{\theta} f_{dist}(\theta) = 2 \sum_{i \in I} \left(\frac{\partial (f_{fk}(\theta))_i}{\partial \theta} \right)^T ((f_{fk}(\theta))_i - G_i) = 0 \Rightarrow$$

$$\Rightarrow \sum_{i \in I} \left(\frac{\partial (f_{fk}(\theta))_i}{\partial \theta} \right)^T ((f_{fk}(\theta))_i - G_i) = 0$$

Запишем формулу для прямой кинематики:

$$J_m(\theta) = \sum_{k=1}^m d_k \begin{pmatrix} \cos(\theta_1 + \theta_2 + \dots + \theta_k) \\ \sin(\theta_1 + \theta_2 + \dots + \theta_k) \end{pmatrix}$$

$$f_{fk}(\theta) = (J_1(\theta), J_2(\theta), \dots, J_n(\theta))$$

Отсюда аналитически выразим Якобиан по столбцам:

$$\frac{\partial J_m(\theta)}{\partial \theta_j} = \begin{cases} \sum_{k=j}^m d_k \begin{pmatrix} -\sin \varphi_k \\ \cos \varphi_k \end{pmatrix}, & j \leq m \\ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, & j > m \end{cases}$$

где $\varphi_k = \sum_{r=1}^k \theta_r = \theta_1 + \theta_2 + \dots + \theta_k$

Для решения задачи будем использовать градиентный спуск.

1.3. Решение на Python

Результат численного решения для начальной и конечной позы манипулятора приведён на рисунке 1.3.

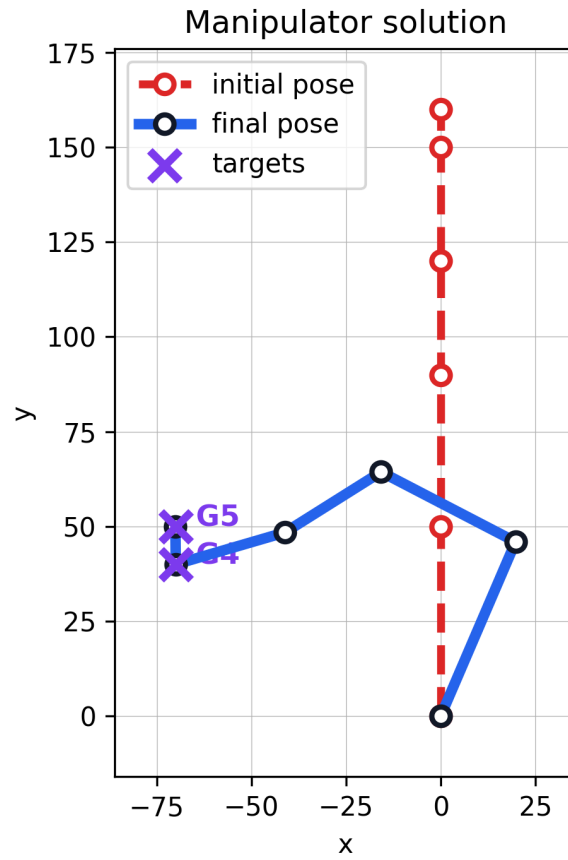


Рисунок 1.3. robot solution

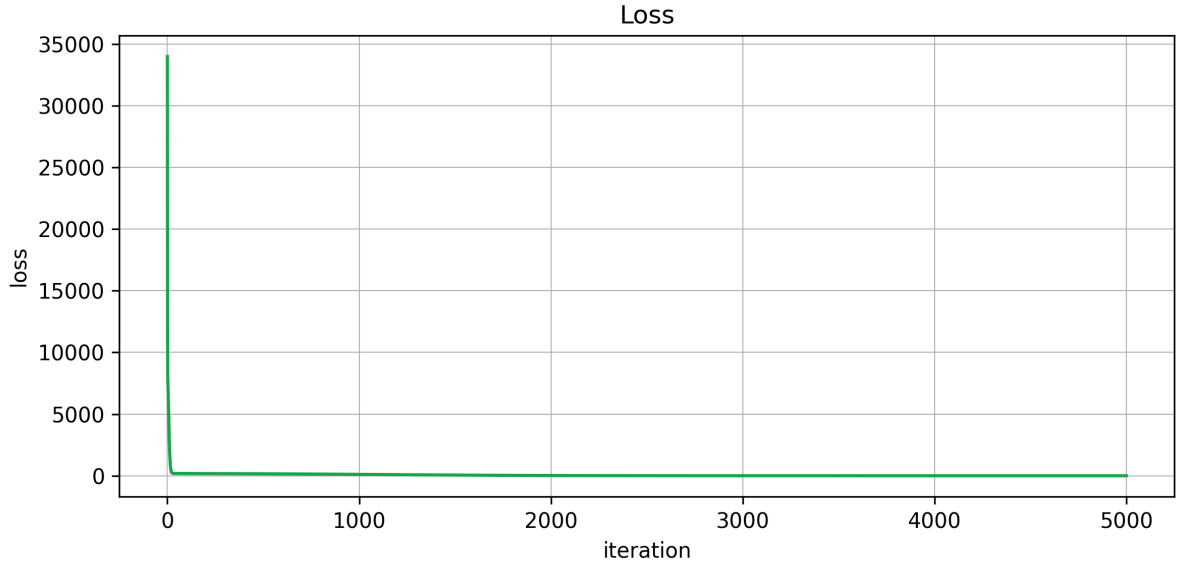


Рисунок 1.4. loss

По графику ошибки на рисунке 1.4 видно, что на начальных итерациях значение функции резко уменьшается, после чего процесс выходит на область малых изменений. Это подтверждает работоспособность аналитического градиента и корректность реализации прямой кинематики.

1.4. Учёт препятствий и проверка безопасности

Препятствия задаются как набор кругов

$$O = \{O_s \mid s = 1, \dots, N_o\}, \quad O_s = (C_s, r_s), \quad C_s = (x_s^o, y_s^o),$$

где C_s - центр препятствия, r_s - радиус препятствия.

Для проверки расстояния от манипулятора до препятствий каждое звено рассматривается как отрезок

$$S_m(\theta) = [J_{m-1}(\theta), J_m(\theta)], \quad J_0 = (0, 0), \quad m = 1, \dots, n.$$

Пусть

$$a = J_{m-1}(\theta), \quad b = J_m(\theta), \quad u = b - a.$$

Ближайшая к центру препятствия точка на отрезке определяется через параметр

$$\tau_{m,s} = \text{clip} \left(\frac{(C_s - a)^T u}{u^T u}, 0, 1 \right),$$

$$P_{m,s} = a + \tau_{m,s} u.$$

Тогда расстояние от центра препятствия до звена равно

$$\text{dist}(C_s, S_m(\theta)) = \|C_s - P_{m,s}\|_2.$$

Минимальный зазор между манипулятором и препятствиями задаётся как

$$\rho(\theta) = \min_{m=1, \dots, n} \min_{s=1, \dots, N_o} (\text{dist}(C_s, S_m(\theta)) - r_s).$$

Конфигурация считается допустимой, если

$$\rho(\theta) \geq r_{safe},$$

где r_{safe} - заданный безопасный зазор. В численном эксперименте используется $r_{safe} = 4$.

Кроме столкновений с внешними препятствиями проверяется самопересечение манипулятора. Для этого вычисляется расстояние между несоседними звеньями. Соседние звенья не сравниваются, так как они имеют общий сустав. Минимальный зазор между несоседними звеньями обозначим как $\rho_{self}(\theta)$. Конфигурация считается безопасной при одновременном выполнении условий

$$\rho(\theta) \geq r_{safe}, \quad \rho_{self}(\theta) \geq r_{self}.$$

В коде эти проверки реализованы функциями `min_clearance`, `min_self_clearance` и `pose_is_safe`.

При переходе от одной конфигурации к другой проверяется не только конечная поза, но и промежуточные положения. Для этого используется функция приведения угла к диапазону $[-\pi, \pi]$:

$$w(\alpha) = ((\alpha + \pi) \bmod 2\pi) - \pi,$$

и кратчайшая разность двух векторов углов:

$$\Delta(\theta^b, \theta^a) = w(\theta^b - \theta^a).$$

Промежуточные положения при переходе задаются как

$$\theta_q = w\left(\theta^a + \frac{q}{Q}\Delta(\theta^b, \theta^a)\right), \quad q = 0, 1, \dots, Q.$$

Переход считается допустимым, если все промежуточные конфигурации безопасны:

$$\min_{q=0, \dots, Q} \rho(\theta_q) \geq r_{safe}, \quad \min_{q=0, \dots, Q} \rho_{self}(\theta_q) \geq r_{self}.$$

В программе для этого используется функция `motion_is_safe`, которая проверяет 25 промежуточных положений между двумя векторами углов.

2. Подготовка данных для нейросетевого оценивания

После построения первого приближения задача расширяется: требуется не только найти конечную позу, но и получить последовательность безопасных промежуточных конфигураций. Для этого в работе используется нейросетевая модель MLP, которая оценивает качество локального кандидатного шага. Перед обучением модели необходимо определить множество возможных конечных конфигураций, способ кодирования входных данных и функцию экспертной оценки.

2.1. Предварительный поиск допустимых конечных конфигураций

Перед построением траектории выполняется вспомогательный поиск допустимых конечных конфигураций. Эти конфигурации не считаются заранее известным решением всей задачи. Они вычисляются численно как возможные конечные положения манипулятора, в которых выбранные суставы достигают целевых точек и сама поза является безопасной.

Обозначим среднюю ошибку достижения целевых точек:

$$E(\theta) = \frac{1}{|I|} \sum_{i \in I} \|J_i(\theta) - G_i\|_2.$$

Множество допустимых целевых конфигураций задаётся как

$$\Theta_G = \{\theta_g \mid E(\theta_g) \leq \varepsilon, \rho(\theta_g) \geq r_{safe}, \rho_{self}(\theta_g) \geq r_{self}\}.$$

Для построения Θ_G несколько раз запускается градиентный спуск из разных начальных приближений:

$$\begin{aligned}\theta_0^{(p)} &\sim U[-\pi, \pi]^n, \\ \theta_g^{(p)} &= \text{GD}(\theta_0^{(p)}, f_{dist}).\end{aligned}$$

После этого сохраняются только те конфигурации, для которых выполняется условие по ошибке и безопасности. В коде этот этап реализован функцией `find_goal_poses`. В демонстрационных сценах используется 60 попыток поиска конечных конфигураций, а допустимая ошибка равна $\varepsilon = 1$.

Важно, что Θ_G не задаёт готовую траекторию. Это только набор возможных конечных поз, найденных решением обратной кинематики. Последующая задача планировщика состоит в том, чтобы безопасно дойти до одной из таких поз из начальной конфигурации.

2.2. Формирование входного вектора MLP

В качестве управляющего воздействия рассматривается изменение всего вектора углов

$$\eta = (\eta_1, \eta_2, \dots, \eta_n) \in \mathbb{R}^n.$$

Кандидатная следующая конфигурация определяется как

$$\theta^+ = w(\theta + \eta),$$

где θ - текущая конфигурация.

Пусть

$$D = \sum_{m=1}^n d_m$$

- суммарная длина звеньев манипулятора. Входной вектор MLP формируется из нормированных текущих углов, нормированного кандидатного изменения углов, координат целевых точек и параметров препятствий:

$$z(\theta, \eta) = \left(\frac{\theta}{\pi}, \frac{\eta}{\pi}, \frac{\text{vec}(G)}{D}, \frac{\text{vec}(O)}{D} \right).$$

Нормирование углов на π , а координат и радиусов на D позволяет использовать в одном входном векторе величины разной природы. Препятствия дополняются фиктивными объектами до фиксированного числа, чтобы размер входного вектора MLP оставался постоянным. В итоговом варианте используется $N_o = 2$, поэтому размер входа равен

$$5 + 5 + 4 + 6 = 20,$$

где первые 5 элементов соответствуют текущим углам, следующие 5 - кандидатному изменению углов, 4 элемента - двум целевым точкам, 6 элементов - двум препятствиям.

2.3. Экспертная оценка кандидатного шага

Для обучения MLP вводится экспертная оценка кандидатного шага. Она строится так, чтобы высокий балл получали шаги, которые уменьшают ошибку достижения цели, приближают конфигурацию к одной из найденных конечных поз, сохраняют безопасный зазор и не являются чрезмерно большими по углам.

Если множество Θ_G непусто, расстояние до ближайшей допустимой конечной конфигурации задаётся как

$$g(\theta^+) = \min_{\theta_g \in \Theta_G} \frac{\|\Delta(\theta_g, \theta^+)\|_2}{n}.$$

Если ни одна допустимая конечная конфигурация не найдена, вместо $g(\theta^+)$ используется ошибка $E(\theta^+)/D$, что позволяет сохранять численную определённую экспертной оценки при генерации обучающих данных.

Штраф за нарушение безопасного расстояния до внешних препятствий:

$$c_{env}(\theta^+) = \frac{\max(0, r_{safe} - \rho(\theta^+))}{D}.$$

Штраф за самопересечение:

$$c_{self}(\theta^+) = \frac{\max(0, r_{self} - \rho_{self}(\theta^+))}{D}.$$

Экспертная оценка, соответствующая программной реализации, имеет вид

$$R(\theta^+, \eta) = -\lambda_E \frac{E(\theta^+)}{D} - \lambda_g g(\theta^+) - \lambda_{env} c_{env}^2(\theta^+) - \lambda_{self} c_{self}^2(\theta^+) - \lambda_\eta |\eta|_2.$$

Здесь λ_E , λ_g , λ_{env} , λ_{self} и λ_η - положительные весовые коэффициенты экспертной оценки. Коэффициент λ_E задаёт значимость ошибки достижения целевых точек. Коэффициент λ_g определяет значимость приближения к одной из найденных допустимых конечных конфигураций. Коэффициент λ_{env} задаёт величину штрафа за приближение к внешним препятствиям ближе допустимого безопасного расстояния. Коэффициент λ_{self} определяет величину штрафа за возможное самопересечение манипулятора. Коэффициент λ_η ограничивает чрезмерно большие изменения углов и тем самым делает предпочтительными более плавные локальные шаги.

Таким образом, задача MLP состоит не в прямой выдаче управляющего воздействия, а в аппроксимации функции качества кандидатного шага.

3. Обучение нейросетевой модели

Обучение MLP выполняется на наборе случайно сформированных сцен. Для каждой сцены задаются целевые точки и препятствия, после чего генерируются случайные текущие конфигурации и кандидатные изменения углов. Для каждого кандидата вычисляется экспертная оценка, описанная в предыдущем разделе. Поэтому обучающая выборка имеет вид набора пар "входной вектор - экспертный балл".

3.1. Формирование обучающей выборки

Для каждой сцены случайно задаются целевые точки, препятствия и кандидатные шаги. Для каждого кандидата строится входной вектор z и вычисляется экспертная оценка R :

$$\begin{aligned}\theta^{(q)} &\sim U[-\pi, \pi]^n, \\ \eta^{(q)} &\sim U[-\eta_{max}, \eta_{max}]^n, \\ \theta_+^{(q)} &= w(\theta^{(q)} + \eta^{(q)}), \\ z^{(q)} &= z(\theta^{(q)}, \eta^{(q)}), \quad y^{(q)} = R(\theta_+^{(q)}, \eta^{(q)}).\end{aligned}$$

MLP аппроксимирует экспертную функцию

$$\hat{R}_\psi(z) \approx R(\theta^+, \eta),$$

а параметры сети находятся из задачи минимизации среднеквадратичной ошибки

$$\psi^* = \arg \min_{\psi} \sum_q \left(\hat{R}_\psi(z^{(q)}) - y^{(q)} \right)^2.$$

3.2. Архитектура и параметры обучения

В итоговом коде используется полносвязная сеть с архитектурой $20 \rightarrow 256 \rightarrow 256 \rightarrow 128 \rightarrow 1$ и функциями активации ReLU между скрытыми слоями. Обучение выполняется в PyTorch с оптимизатором Adam. Перед обучением входы и целевые значения нормализуются по среднему и стандартному отклонению. Эти параметры нормализации сохраняются вместе с весами сети, что необходимо для корректного последующего использования модели.

Параметры численного эксперимента приведены в таблице [3.1](#).

Таблица 3.1

Параметр	Значение
Длины звеньев	[50, 40, 30, 20, 10]
Начальная конфигурация	$(\pi/2, 0, 0, 0, 0)$
Целевые индексы	[3, 4]
Число случайных сцен	1000
Число примеров на сцену	2000
Общий размер обучающей выборки	2 000 000 примеров
Размер входного вектора	20
Число препятствий в сцене	2
Безопасный зазор r_{safe}	4
Допуск по ошибке ε	1
Максимальный шаг η_{max}	0.3 рад
Архитектура MLP	(256, 256, 128)
Число эпох	80
Размер батча	8192
Оптимизатор	Adam

3.3. Результаты обучения

В ходе запуска было сформировано 2 000 000 обучающих примеров. Генерация случайных сцен выполнялась параллельно с использованием backend `loky` и 24 рабочих процессов. Время генерации сцен составило 253.8 с, время обучения MLP - 27.8 с, общее время конвейера обучения - 281.5 с. Финальное значение функции потерь MSE составило 0.06822.

Полученное значение ошибки используется как численный показатель того, насколько обученная сеть приблизилась экспертной функции на сформированной выборке. После обучения веса модели и параметры нормализации сохраняются, поэтому одна и та же обученная MLP может применяться при последующем решении демонстрационных сцен без повторного обучения.

4. Планирование движения с использованием MLP

После обучения MLP используется как оценщик качества локальных шагов. В отличие от прямого расчёта конечной траектории, планирование выполняется итерационно: в текущей конфигурации формируется набор кандидатных изменений углов, затем из него удаляются небезопасные переходы, а оставшиеся варианты оцениваются нейросетью и экспертной функцией.

4.1. Генерация и фильтрация кандидатных шагов

На каждом шаге движения формируется множество кандидатных изменений углов

$$\mathcal{H}(\theta) = \{\eta_k \mid k = 1, \dots, K\}.$$

Если множество допустимых конечных конфигураций Θ_G непусто, дополнительно строится кандидатный шаг в сторону ближайшей конечной конфигурации:

$$\theta_g^* = \arg \min_{\theta_g \in \Theta_G} \|\Delta(\theta_g, \theta)\|_2,$$

$$\eta_0 = \text{clip}(\Delta(\theta_g^*, \theta), -\eta_{max}, \eta_{max}).$$

Остальные кандидаты генерируются случайно:

$$\eta_k \sim U[-\eta_{max}, \eta_{max}]^n.$$

Для каждого кандидата проверяется допустимость всего перехода:

$$\begin{aligned} \theta_k^+ &= w(\theta + \eta_k), \\ \min_{q=0, \dots, Q} \rho \left(w \left(\theta + \frac{q}{Q} \eta_k \right) \right) &\geq r_{safe}. \end{aligned}$$

Остаётся только множество безопасных кандидатов

$$\mathcal{H}_{safe}(\theta) = \{\eta_k \in \mathcal{H}(\theta) \mid \theta \rightarrow w(\theta + \eta_k) \text{ допустим}\}.$$

4.2. Выбор шага и критерий остановки

Для каждого безопасного кандидата вычисляются две оценки: предсказанная нейросетью $\hat{R}_\psi$ и фактическая экспертная оценка R . Лучший шаг выбирается по максимуму суммарного критерия

$$\eta^* = \arg \max_{\eta_k \in \mathcal{H}_{safe}(\theta)} \left(0.5 \hat{R}_\psi(z(\theta, \eta_k)) + 0.5 R(w(\theta + \eta_k), \eta_k) \right).$$

После выбора шага выполняется обновление

$$\theta_{t+1} = w(\theta_t + \eta^*).$$

Итерации продолжаются до выполнения условия

$$E(\theta_t) \leq \varepsilon$$

или до достижения максимального числа шагов T_{max} . В демонстрационном запуске использовались $K = 20$ кандидатов на шаг и $T_{max} = 500$.

Такой способ планирования сохраняет связь с аналитической постановкой задачи: MLP не заменяет проверку безопасности и не отменяет функцию ошибки, а используется как дополнительный оценщик, позволяющий ранжировать локальные варианты движения.

5. Демонстрационные сцены и оценка результатов

Проверка обученной модели выполнялась на девяти демонстрационных сценах. В каждой сцене заданы две целевые точки для выбранных сочленений и два круглых препятствия. Для каждой сцены сначала находится набор допустимых конечных конфигураций, затем запускается пошаговый планировщик с использованием MLP.

5.1. Визуальная проверка траекторий

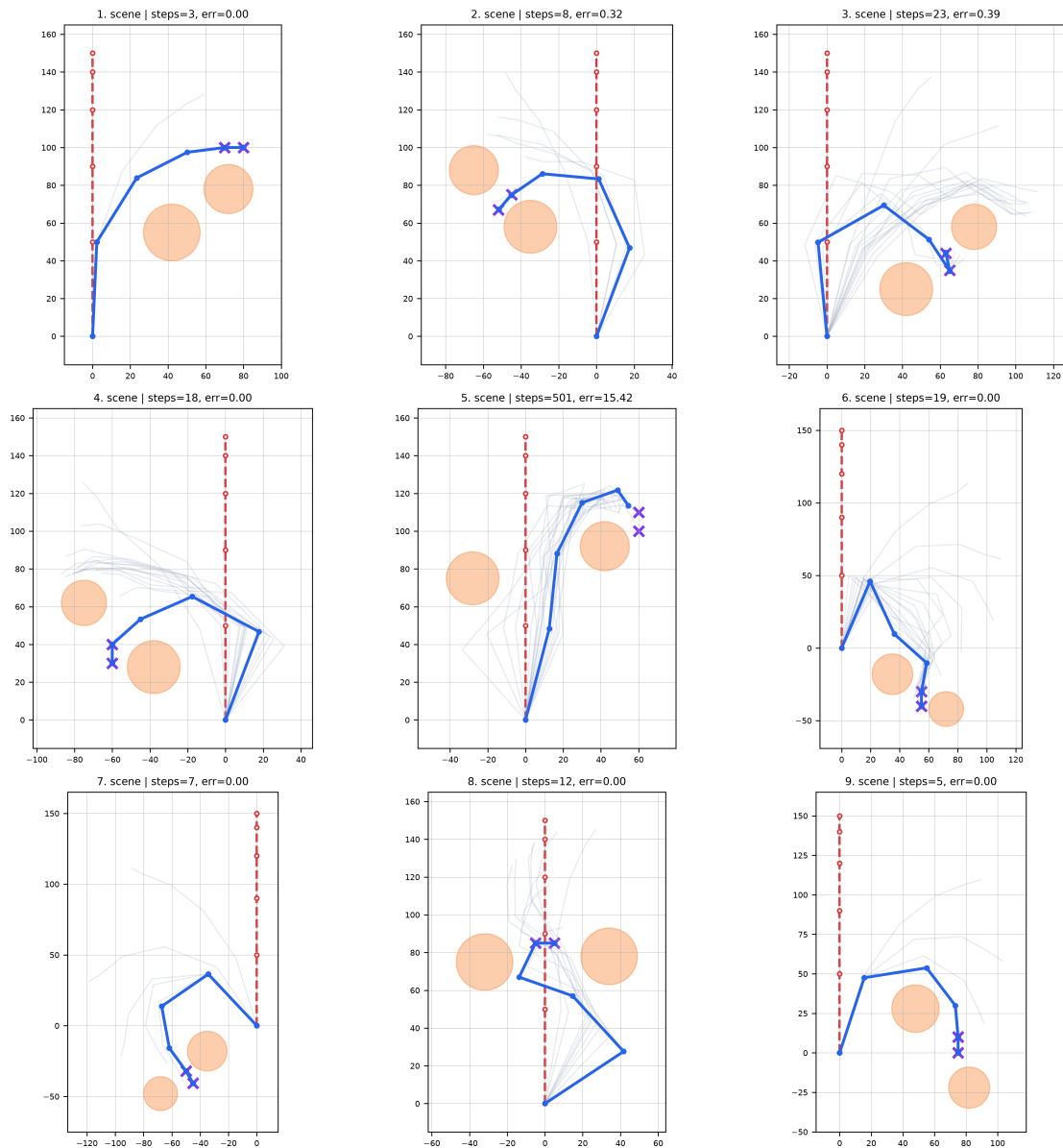


Рисунок 5.1. Результаты планирования движения для девяти демонстрационных сцен

На рисунке 5.1 красной пунктирной линией показана начальная поза манипулятора, синей линией - конечная поза, серыми линиями - промежуточные конфигурации траектории, фиолетовыми крестами - целевые точки, оранжевыми кругами - препятствия. Такой вид графика позволяет одновременно проверить достижение целей и отсутствие пересечения с препятствиями.

По результатам запуска планировщик достиг целевых точек в 8 из 9 демонстрационных сцен. В 5 сцене движение завершилось из-за недостижимости целевой конфигурации в силу того, что робот должен сохранять определенное безопасное расстояние до препятствий.

5.2. Диагностика ошибки и безопасных зазоров

Для более подробного анализа дополнительно построены графики изменения ошибки и зазоров по шагам. На этих графиках для каждой сцены отображаются $E(\theta_t)$, минимальный зазор до препятствий $\rho(\theta_t)$ и минимальный зазор между несоседними звеньями $\rho_{self}(\theta_t)$.

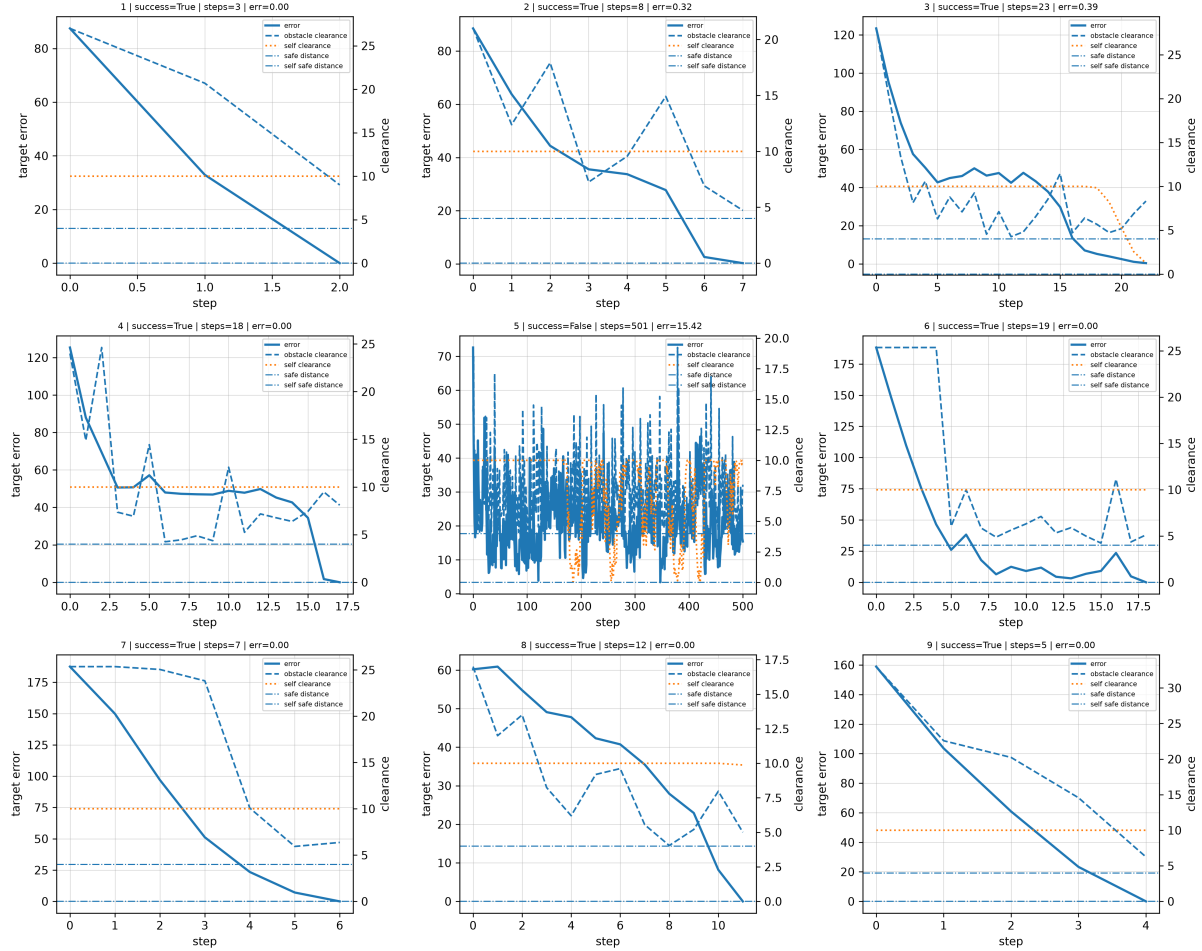


Рисунок 5.2. Изменение ошибки и минимального зазора по шагам для девяти сцен

Рисунок 5.2 используется для проверки не только конечного результата, но и всего процесса движения. Если ошибка уменьшается, а значения зазоров не опускаются ниже заданных порогов, то траектория считается согласованной с поставленными ограничениями. Для неуспешной сцены график показывает, что ограничение по числу шагов было достигнуто раньше, чем ошибка стала меньше допуска ε .

Заключение

В ходе работы была реализована модель плоского многозвенового манипулятора и алгоритм управления, сочетающий аналитическую кинематику, градиентный спуск, проверку столкновений и нейросетевую оценку кандидатных шагов. Прямая кинематика используется для вычисления координат суставов по углам, а аналитический якобиан - для решения задачи обратной кинематики методом градиентного спуска.

Для учёта препятствий манипулятор рассматривается как набор отрезков, а препятствия - как окружности. Безопасность позы определяется минимальным зазором от звеньев до препятствий и минимальным зазором между несоседними звеньями. Дополнительно проверяется безопасность всего перехода между двумя позами через дискретизацию промежуточных конфигураций.

Нейросетевая модель MLP обучалась не выдавать управляющее действие напрямую, а аппроксимировать экспертную оценку качества кандидатного шага. Такой подход позволяет на каждом шаге формировать набор возможных изменений углов, отбрасывать небезопасные варианты и выбирать лучший из оставшихся по суммарной оценке MLP и аналитической экспертной функции.

По результатам демонстрационного эксперимента обученная модель успешно решила сцены, за исключением той, где намеренно была поставлена невыполнимая задача, чтобы увидеть наглядно, как робот будет себя вести в такой ситуации.